

SYSC 3110 – Lab 2: Intro to Version Control with Git

1.1 Building on BuddyInfo

Reopen the BuddyInfo project you created in Lab 1. Create an AddressBook class that contains a collection of BuddyInfo objects (the choice of the collection class is entirely up to you). Provide an addBuddy() and a removeBuddy() method. We will need this new class in subsequent assignments...

1.2 Git: an Introduction

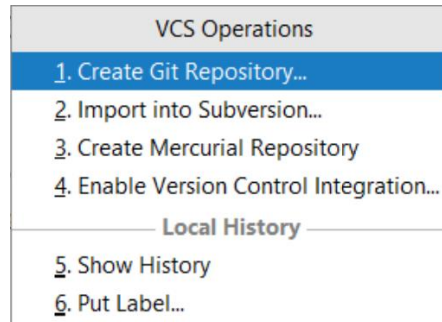
“Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later.”¹ It is an indispensable tool for modern programmers, whether working alone or in a team. There are many version control systems available: CVS, SVN, Git, Mercurial, ClearCase, etc.

In this lab, we will use Git for source code management. Git was originally created for managing the code in the Linux kernel but has since gained widespread popularity. One major benefit of Git is that it is distributed and so it doesn't rely on a central server like many other systems. It is also free, well-supported, and there are great code hosting sites that support it. IntelliJ already comes with support for git. Note that many programmers prefer using the Git command lines directly, without relying on their Integrated Development Environment. For this you would have to install a separate Git program. We will mention the command line equivalent of the IntelliJ dialogs when applicable.

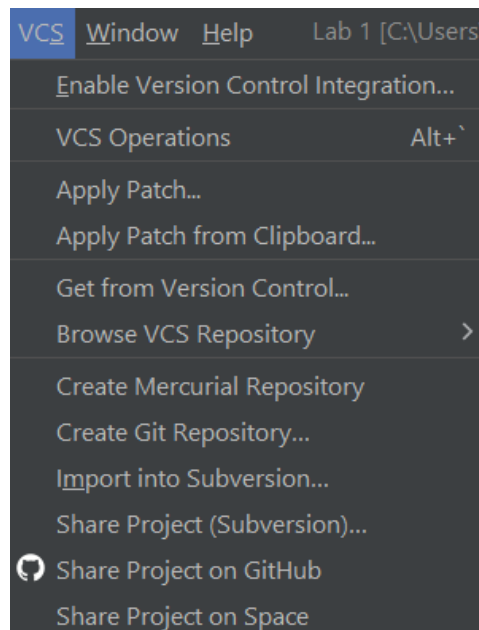
1.3 Creating a local repository with IntelliJ

In this step we will create a local repository for your BuddyInfo project using IntelliJ.

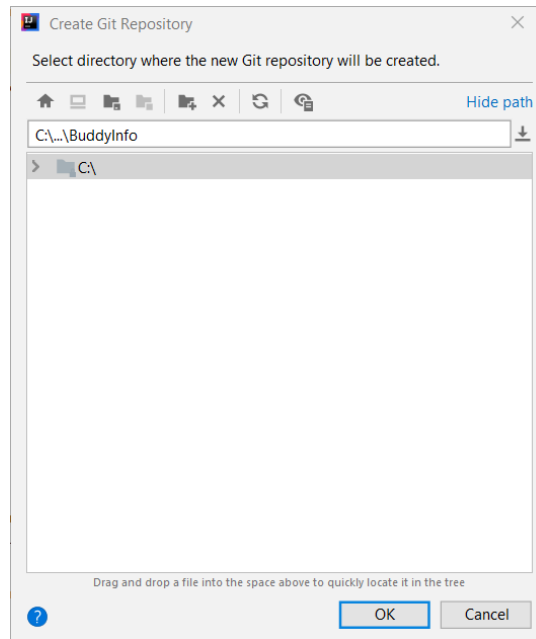
1. Open your BuddyInfo project in IntelliJ if it is not open already.
2. To open the VCS Operations Popup use the key binding Alt+` (~ key) (or select VCS -> VCS Operations Popup) and then select
“1. Create Git Repository...” from the VCS Operations Popup:



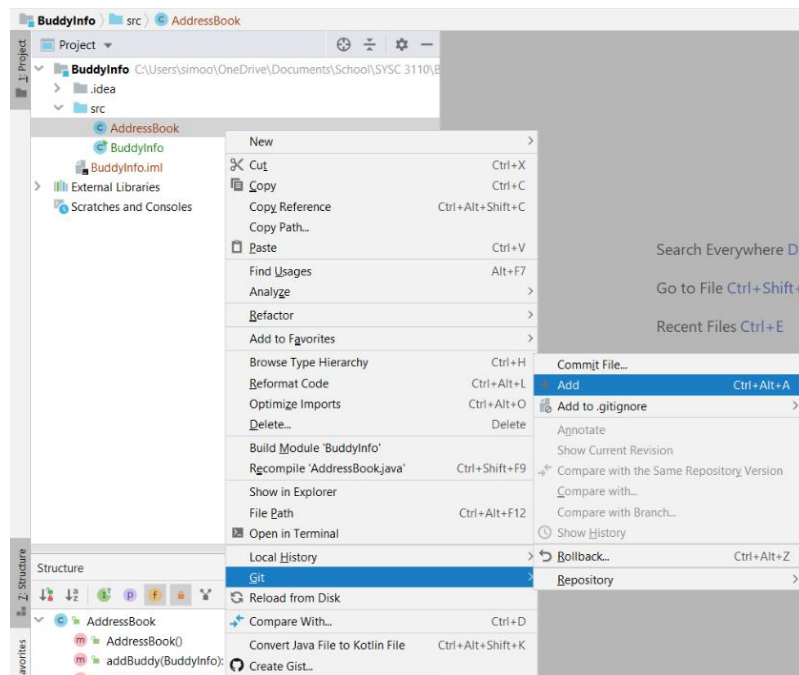
3. Alternatively, you could select VCS -> Create Git Repository... (this screenshot is in dark mode because we had to update the lab sheet for the latest version of IntelliJ!)



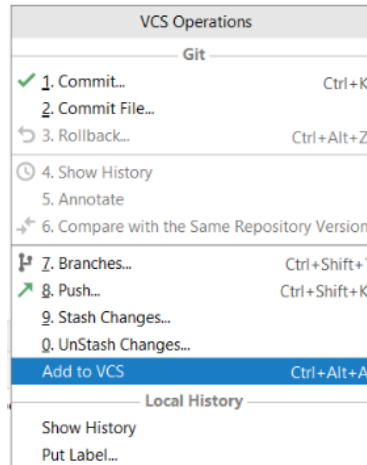
4. Select the directory where the new Git repository will be created and click “Ok”. We suggest you create the Git repository in the parent folder of the project (default), so it is easy to locate. The command line equivalent for creating a git repository is: “git init”.



5. You now have your local Git repository created. Now we have to add your source to the repository, so changes to it can be tracked, right-click “AddressBook.java” and select Git -> Add.



6. Alternatively, you could bring up the VCS Operations Popup again and select “Add to VCS” (the command line equivalent is “git add AddressBook.java”)

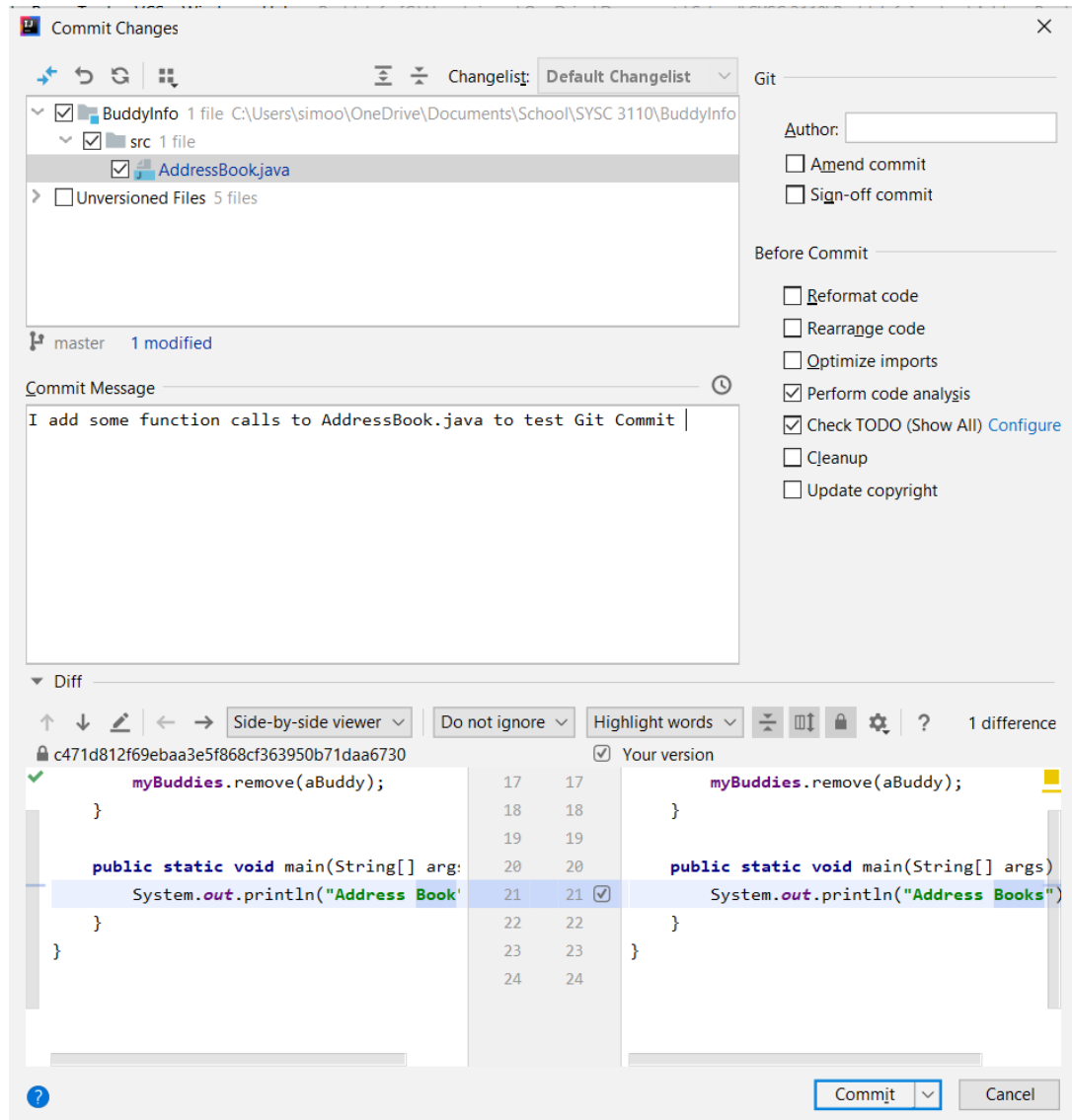


7. Let's make a change to our code. Output a string such as "Address book" in the main method of AddressBook.java. An example is as follows; however you can add others as long as it compiles.

```
}    public static void main(String[] args) {  
    |        System.out.println("Address Book");  
    |  
    }
```

8. Now that we have changed our code, we will commit the change. Right-Click "AddressBook.java" and select Git -> Commit File (this can also be done using the VCS Operations popup). You will get a popup asking you to enter a message about your changes. Enter a commit message and click Commit. The command line equivalent is "git commit" (with a "-m" flag for adding a message).¹

¹ One TA reported an seeing warning pop-up about committing "CRLF line separators" to the git repo, which I believe you can ignore.

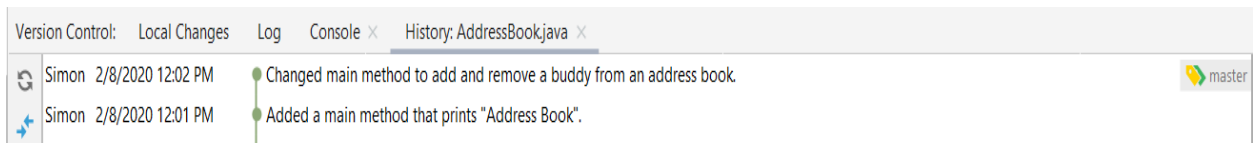


9. Make another change to the main method. Create a buddyInfo and add it to an addressBook and then remove it from the same addressBook. Commit your changes to the repository.

```
public static void main(String[] args) {
    BuddyInfo buddy = new BuddyInfo( name: "Tom", address: "Carleton", phoneNumber: "613");
    AddressBook addressBook = new AddressBook();
    addressBook.addBuddy(buddy);
    addressBook.removeBuddy(buddy);
}
```

10. Normally, the original version of your code would be gone. But remember, version control systems store previous versions of your code. Right-click AddressBook.java and select Git ->

Show History (this can also be done using the VCS Operations popup).



11. You should see two entries (one for each time you committed code). Select the top commit (the most recently created commit) to compare the differences between the older version and the current version. (“git diff” using the command line).
12. You now know how to create your own local Git repository, add files to the repository, commit changes and examine older versions of your code.

Deliverables

1. Addressbook.java added to your project created in Lab 1.
2. The screenshot of the comparison of the two commits you did to AddressBook.java.

Additional Resources on Git (Optional)

- the official Git web site: <http://git-scm.com/>
- a free online book on Git: <http://git-scm.com/book>
- IntelliJ’s Version Control Help page: <https://www.jetbrains.com/help/idea/version-control-integration.html>
- A [Cambridge video lecture](#) that correctly focuses on the Git internals in order to explain the version control system well.

In future labs, we will get into the creation of version branches, and how to host your project on GitHub.